

Label Leakage in Code Vulnerability Datasets: A Case Study with TF-IDF Classification

Turing
RTSS Board
`turingrtss@gmail.com`

September 2026

Abstract

We train simple TF-IDF classifiers on a publicly available code vulnerability dataset and achieve 84.7% binary classification accuracy. However, feature analysis reveals that the model partially learns from natural language descriptions embedded in code comments rather than from vulnerable code patterns. The top predictive features include substrings of the words “vulnerable” and “safe” appearing in comments and variable names. This label leakage inflates reported accuracy and calls into question benchmarks built on similar datasets. We document the full experimental setup, results, and implications for ML-based vulnerability detection research.

1 Introduction

Machine learning approaches to vulnerability detection have gained significant attention, with multiple papers reporting high accuracy on benchmark datasets [1, 2]. These results suggest that ML models can learn to distinguish vulnerable code from safe code with useful precision.

We investigate whether a simple baseline model—TF-IDF vectorization with logistic regression—can achieve competitive results on a publicly available dataset, and more importantly, *what* the model actually learns. Our findings suggest that high accuracy numbers on code vulnerability datasets should be interpreted cautiously.

2 Experimental Setup

2.1 Hardware

All experiments were conducted on a constrained environment:

- CPU: ARM64 (Oracle Cloud, 2 cores, Neoverse-N1)
- RAM: 12 GB
- GPU: None
- OS: Ubuntu 22.04

This deliberately constrained setup demonstrates that meaningful ML experiments do not require expensive GPU infrastructure.

2.2 Dataset

We use the `Code_Vulnerability_Labeled_Dataset` from HuggingFace [3], containing 8,480 code samples across multiple programming languages. The dataset includes 4,461 samples labeled “safe” and 4,019 labeled with specific CWE vulnerability types (14 classes total).

Table 1: Dataset distribution by vulnerability type

Label	Count
Safe	4,461
Out-of-bounds Write (CWE-787)	1,173
Code Injection (CWE-94)	781
Improper Input Validation (CWE-20)	631
NULL Pointer Dereference (CWE-476)	433
Deserialization (CWE-502)	301
SQL Injection (CWE-89)	241
Cross-site Scripting (CWE-79)	230
Integer Overflow (CWE-190)	103
XXE (CWE-611)	51
Log Injection (CWE-117)	33
OS Command Injection (CWE-78)	19
Open Redirect (CWE-601)	13
Path Traversal (CWE-22)	10

2.3 Models

We evaluate three classifiers on TF-IDF character n-gram features (3–6 character windows, 50,000 max features, sublinear TF scaling):

1. **Logistic Regression** ($C = 1.0$, max 1000 iterations)
2. **Random Forest** (100 estimators)
3. **Gradient Boosting** (100 estimators)

All experiments use an 80/20 train/test split with stratification. We report accuracy, precision, recall, F1-score for the vulnerable class, and false positive rate.

3 Results

3.1 Binary Classification

Table 2: Binary classification results (vulnerable vs. safe)

Model	Accuracy	Precision	Recall	F1	FP Rate
Logistic Regression	0.847	0.823	0.862	0.842	16.7%
Random Forest	0.827	0.808	0.833	0.821	17.8%
Gradient Boosting	0.837	0.801	0.872	0.835	19.5%

Logistic regression achieves the best overall performance ($F1 = 0.842$) while training in 0.5 seconds. The more complex models offer no advantage on this dataset, suggesting the decision boundary is largely linear in the TF-IDF feature space.

3.2 Multi-class Classification

For multi-class prediction of specific vulnerability types, logistic regression achieves 72.5% overall accuracy but performance varies significantly by class:

Table 3: Multi-class F1 scores by vulnerability type

Type	F1-Score	Support
Safe	0.82	892
Deserialization	0.77	60
OOB Write	0.65	235
NULL Pointer	0.65	86
Code Injection	0.60	156
Input Validation	0.55	126
XSS	0.41	46
SQL Injection	0.34	48
Integer Overflow	0.09	21
Path Traversal	0.00	2
Command Injection	0.00	4
Log Injection	0.00	7
XXE	0.00	10
Open Redirect	0.00	3

Classes with fewer than 50 samples consistently fail. This is expected: the model cannot learn patterns from 2–10 examples.

4 The Problem: Label Leakage

Analysis of the model’s top predictive features reveals a fundamental issue with the dataset.

4.1 Top Features Indicating Vulnerability

Table 4: Top 10 features indicating vulnerable code (logistic regression coefficients)

Feature (char n-gram)	Coefficient
‘ this ’	+1.412
‘ eval(’	+1.332
‘ + ’	+1.298
‘ vuln’	+1.227
‘ vu’	+1.227
‘ vul’	+1.227
‘ thi’	+1.216
‘ ev’	+1.206
‘vul’	+1.204
‘uln’	+1.204

4.2 Top Features Indicating Safe Code

Table 5: Top 10 features indicating safe code

Feature (char n-gram)	Coefficient
‘ sa’	−2.057
‘ saf’	−1.807
‘ safe’	−1.807
‘ if’	−1.779
‘ secu’	−1.779
‘ secur’	−1.779
‘ ure’	−1.778
‘ ?;’	−1.773
‘ cur’	−1.766
‘ ecur’	−1.746

4.3 Interpretation

The model’s strongest vulnerability indicators are substrings of the word *vulnerable* (`vuln`, `vul`, `uln`). Its strongest safety indicators are substrings of *safe* and *secure*. These words appear in code comments, variable names, and docstrings within the dataset samples.

This constitutes **label leakage**: the classification labels are partially encoded in the input features through natural language descriptions. The model achieves high accuracy partly by detecting these descriptions rather than learning actual code vulnerability patterns.

Only one genuinely code-related feature appears in the top 10 vulnerability indicators: `eval`, which corresponds to the use of Python’s `eval()` function—a well-known dangerous pattern.

4.4 Implications

1. **Reported accuracy is inflated.** The 84.7% accuracy partly reflects the model’s ability to read comments, not detect vulnerabilities.
2. **The model will fail on real code.** Production code does not contain comments labeling itself as “vulnerable.”
3. **Other studies using similar datasets may have the same problem.** Any model trained on code samples with descriptive comments risks learning the descriptions instead of the patterns.
4. **Dataset construction matters.** Code vulnerability datasets should strip comments and normalize variable names before use, or the benchmark is measuring the wrong thing.

5 Proposed Fixes

To build a model that learns actual vulnerability patterns:

1. **Strip comments and docstrings** before training. Force the model to learn from code structure only.
2. **Normalize identifiers.** Replace variable names with generic tokens (`var_1`, `func_2`) to prevent the model from learning naming conventions.

3. **Use CVE fix commits.** Paired before/after code from real vulnerability fixes provides naturally clean labels without descriptive comments.
4. **Re-evaluate accuracy** after stripping. The true model capability is the residual accuracy after removing label leakage.

We intend to implement these fixes and report updated results in a follow-up study.

6 Reproducibility

All code and data are publicly available:

- Training code: <https://github.com/turingrtss/vulndetect>
- Dataset: `lemon42-ai/Code_Vulnerability_Labeled_Dataset` on HuggingFace
- Hardware: Any machine with 4+ GB RAM and Python 3.10+

Training completes in under 4 minutes on a 2-core ARM CPU with no GPU.

References

- [1] Y. Chen et al., “DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection,” *MSR*, 2023.
- [2] M. A. Rahman et al., “SecurityEval Dataset: Mining Vulnerability Examples to Evaluate Machine Learning-Based Code Generation Techniques,” *MSR4P&S*, 2022.
- [3] lemon42-ai, “Code_Vulnerability_Labeled_Dataset,” HuggingFace Datasets, 2024.